

Safe FSM In Libero

Course Workbook

Table of Contents

About this Workbook

[Slide 3](#)

Pre-Lab: Workshop Pre-requisites

[Slide 4](#)

Lab : Understanding Vivado

[Slide 7](#)

About this Workbook

This workbook is designed to be used in conjunction with the Safe FSM in Libero workbook.

The contents of this workbook are created by Aduvo Engineering & Training, Ltd.

If you have any questions about the contents, or need assistance, please contact Adam Taylor at adam@adiuvoengineering.com.

Pre-Lab

Workshop Pre-requisites

Required Hardware

There is no required hardware for this course.

Downloads and Installations

Step 1 – Download and install the following at least one day prior to the workshop. This may take a significant amount of time and drive space.

Libero SoC 11.9	Download
Source Project Files	Download

Lab

Overview and Introduction to Libero

Libero FSM: Overview and Introduction to Libero

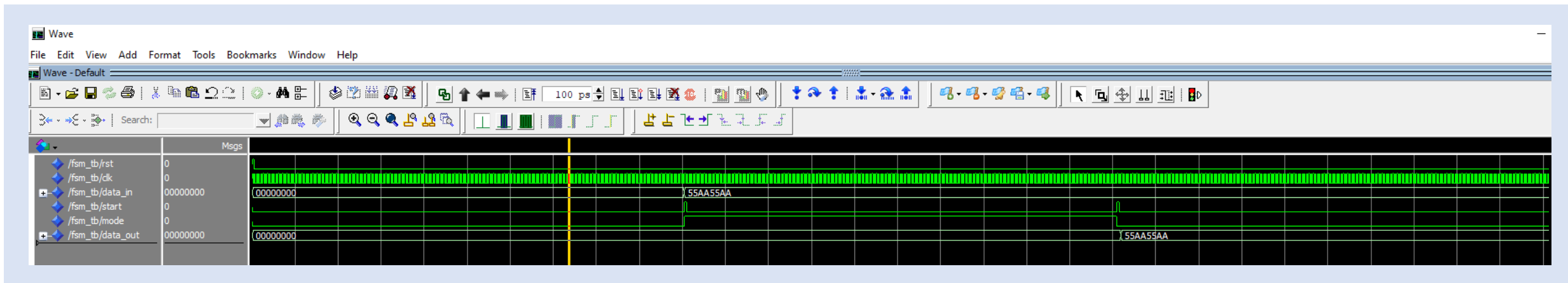
Step 1 – Create a new Libero Project – Targeting a RT ProASIC3E Device



Libero FSM: Overview and Introduction to Libero

Step 2 – To the new project add in the example files fsm_full_case.vhd and fsm_tb.vhd – Set the fsm_tb as stimulus and run the simulation opening ModelSim to run interactively.

Note the state machines behavior performing read and writes



Libero FSM: Overview and Introduction to Libero

Step 3 – In your chosen editor open the file fsm_full_case.vhd – Notice the following

```
architecture rtl of fsm is
  --attribute syn_safe_case      : string;
  --attribute syn_safe_case of rtl : architecture is "true";
  type state is (idle, rd, wr, unmapped);

  signal current_state : state;
  signal transfer_word : std_logic_vector(31 downto 0);
begin

  fsm : process (rst, clk)
  begin
    if rst = '1' then
      current_state <= idle;
      transfer_word <= (others => '0');
      data_out      <= (others => '0');
    elsif rising_edge(clk) then
      case current_state is
        when idle =>
          if start = '1' then
            if mode = '0' then
              current_state <= rd;
            else
              current_state <= wr;
            end if;
          end if;
        when rd =>
          data_out      <= transfer_word;
          current_state <= idle;
        when wr =>
          transfer_word <= data_in;
          current_state <= idle;
        when others =>
          current_state <= idle;
          --null;
        end case;
      end if;
    end process;
end architecture;
```

Synthesis Attributes – Currently commented out
State Machine with three declared states – Not power of 2 states used

Single Process State Machine

When Others defines action

Libero FSM: Overview and Introduction to Libero

Step 4 – Open Synplify Pro interactively and run the Synthesis, once complete open the log

The screenshot displays the Synplify Pro software interface. The 'Run' button in the left sidebar is highlighted with a red rectangle. The main window shows the 'Project Status' tab, which includes a 'Project Settings' table, a 'Run Status' table, an 'Area Summary' table, a 'Timing Summary' table, and an 'Optimizations Summary' table. The 'Run Status' table shows the completion of the synthesis process. The 'Area Summary' table shows 71 Core Cells and 68 IO Cells. The 'Timing Summary' table shows a clock frequency of 100.0 MHz and a slack of 5.067. The 'Optimizations Summary' table shows 1 / 0 more optimizations. The bottom status bar indicates the completion of the synthesis process.

Project Settings

Project Name	fsm_syn	Device Name	synthesis: Microsemi ProASIC3L : RT3PE3000L
Implementation Name	synthesis	Top Module	work fsm
Retiming	0	Resource Sharing	1
Fanout Guide	24	Disable I/O Insertion	0
Disable Sequential Optimizations	0	FSM Compiler	1

Run Status

Job Name	Status	CPU Time	Real Time	Memory	Date/Time
Compile Input (compiler)	Complete	0	00m:01s	-	02/12/2020 12:42:42
Pre-mapping (premap)	Complete	0	0m:00s	110MB	02/12/2020 12:42:43
Map & Optimize (fpga_mapper)	Complete	0	0m:00s	110MB	02/12/2020 12:42:44

Area Summary

Core Cells	IO Cells
71	68

Timing Summary

Clock Name (clock_name)	Req Freq (req_freq)	Est Freq (est_freq)	Slack (slack)
fsmclk	100.0 MHz	202.7 MHz	5.067

Optimizations Summary

Combined Clock Conversion	1 / 0 more

Information

```
Return Code: 1
Run Time: 00h:00m:00s
Complete: Map on fsm_syn|synthesis
Complete: Logic Synthesis on fsm_syn|synthesis
0
Note: Opening local project RTL file 'C:\Users\aptay\Dropbox\adiuvo\mission_critical_course\RTL\Synplify_Unsafe\synthesis\fsm.srs'
```

Libero FSM: Overview and Introduction to Libero

Step 5 – Examine the log – Notice the state machine has 3 reachable states

```
#Build: Synplify Pro L-2016.09M-SP1-5, Build 264R, Oct 26 2017
#install: C:\Microsemi\Libero_SoC_v11.9\SynplifyPro
#OS: Windows 8 6.2
#Hostname: DESKTOP-L3OMJCL

# Wed Dec 02 12:42:41 2020

#Implementation: synthesis

Synopsys HDL Compiler, version comp2016q4pl, Build 512R, built Oct 26 2017
@N: : | Running in 64-bit mode
Copyright (C) 1994-2016 Synopsys, Inc. All rights reserved. This Synopsys software and all associated documentation are proprietary

Synopsys VHDL Compiler, version comp2016q4pl, Build 512R, built Oct 26 2017
@N: : | Running in 64-bit mode
Copyright (C) 1994-2016 Synopsys, Inc. All rights reserved. This Synopsys software and all associated documentation are proprietary

@N:CD720 : std.vhd(146) | Setting time resolution to ps
@N: : fsm_full_case.vhd(47) | Top entity is set to fsm.
VHDL syntax check successful!
@N:CD231 : std1164.vhd(890) | Using onehot encoding for type mv19plus. For example, enumeration 'U' is mapped to "1000000000".
@N:CD630 : fsm_full_case.vhd(47) | Synthesizing work.fsm.rtl.
@N:CD233 : fsm_full_case.vhd(65) | Using sequential encoding for type state.
Post processing for work fsm rtl
@N:CL201 : fsm_full_case.vhd(73) | Trying to extract state machine for register current_state.
Extracted state machine for register current_state
State machine has 3 reachable states with original encodings of:
    00
    01
    10

At c_vhdl Exit (Real Time elapsed 0h:00m:00s; CPU Time elapsed 0h:00m:00s; Memory used current: 72MB peak: 73MB)
```

Libero FSM: Overview and Introduction to Libero

Step 6 – Examine the log – Notice the state machine registers are replicated – Potential for FSM SEU issues, if replicated registers are corrupted.

```
High Fanout Net Report
*****
Driver Instance / Pin Name      Fanout, notes
-----
current_state[0] / Q            33
current_state[1] / Q            33
=====
@N:FP130 : | Promoting Net rst_c on CLKBUF rst_pad
@N:FP130 : | Promoting Net clk_c on CLKBUF clk_pad

Finished technology timing optimizations and critical path resynthesis (Real Time elapsed 0h:00m:00s; CPU Time elapsed 0h:00m:00s; Memory used current: 109MB peak: 110MB)

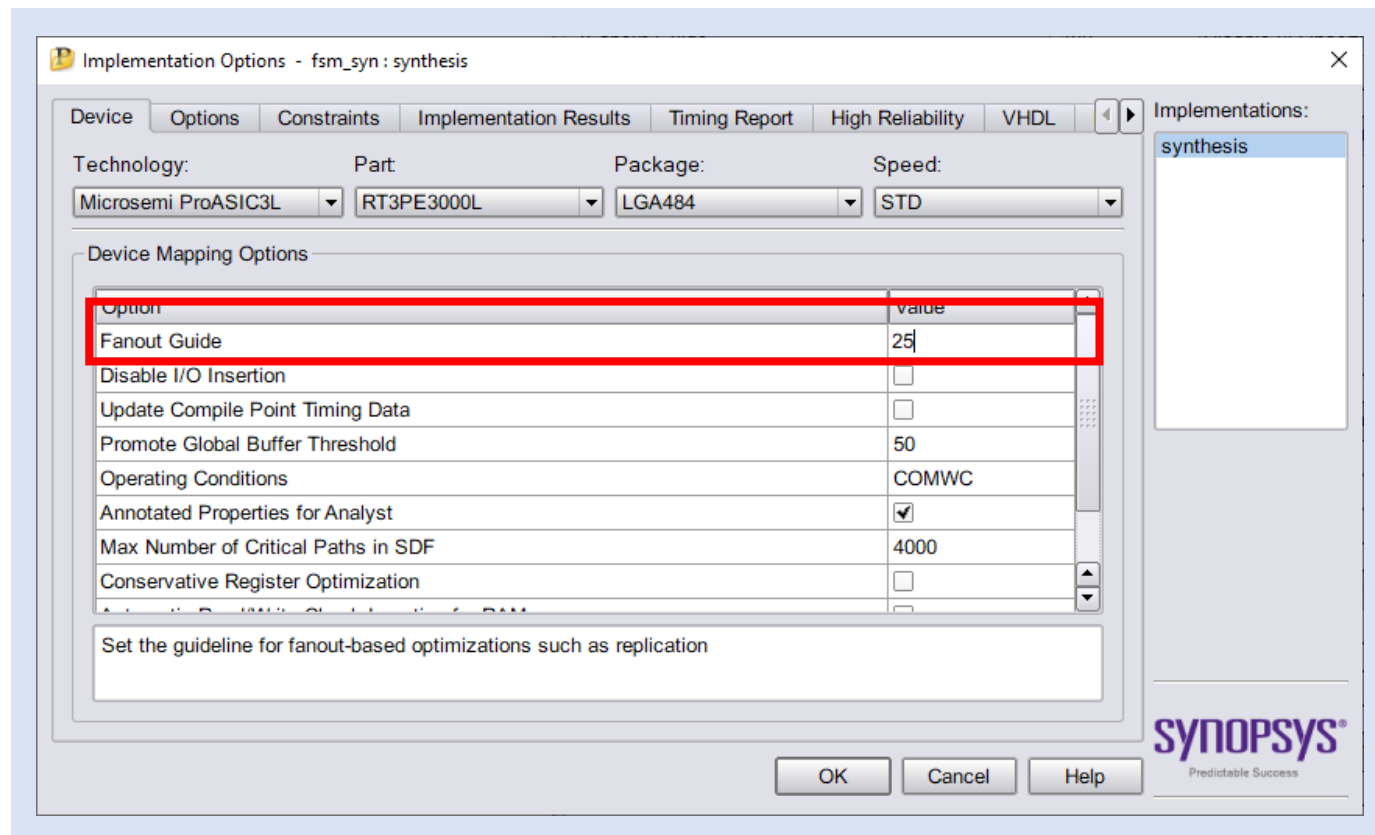
Problem in opening file "C:\Users\aptay\Dropbox\adiuvo\mission_critical_course\RTL\Synplify_Unsafe\synthesis\synlog\report\fsm_fpga_mapper_runstatus.xml" for writing. To create
Replicating Sequential Instance current_state[1], fanout 33 segments 2
Replicating Sequential Instance current_state[0], fanout 33 segments 2

Added 0 Buffers
Added 2 Cells via replication
    Added 2 Sequential Cells via replication
    Added 0 Combinational Cells via replication

Finished restoring hierarchy (Real Time elapsed 0h:00m:00s; CPU Time elapsed 0h:00m:00s; Memory used current: 109MB peak: 110MB)
```

Libero FSM: Overview and Introduction to Libero

Step 7 – We can stop replication, it required by changing the Fan Out Parameter in the Implementation options – However, this still makes leaves the design unsafe due to uncovered states



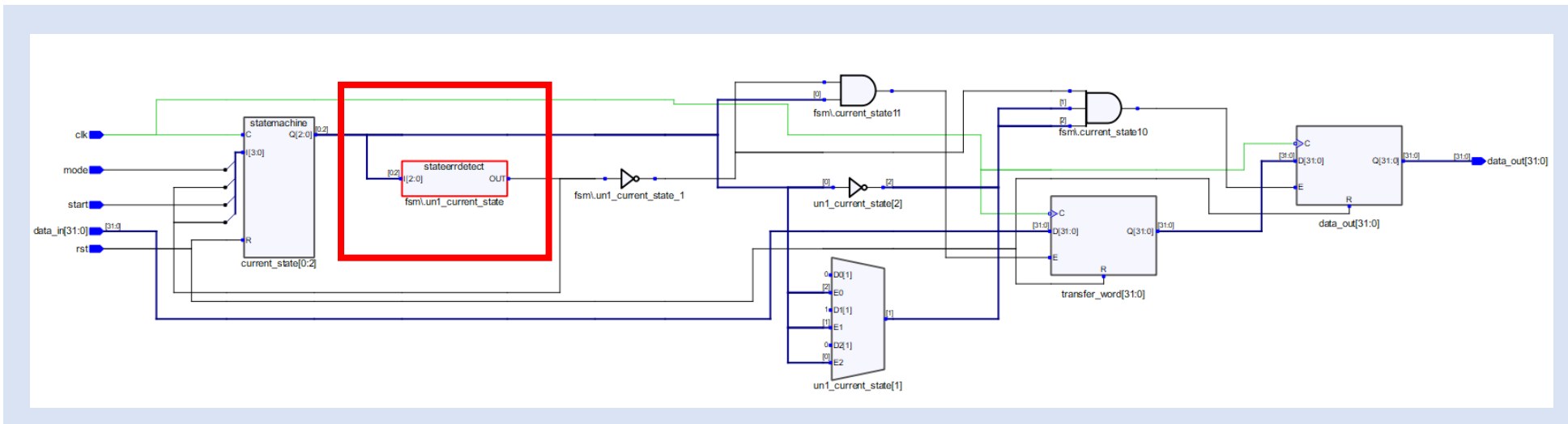
Libero FSM: Overview and Introduction to Libero

Step 8 – Uncomment the attribute lines in the VHDL and re run the synthesis, open the log file and inspect the notification. Notice the Safe Case FSM being implemented.

```
Finished Pre Mapping Phase.  
@N:BN225 : | Writing default property annotation file C:\Users\aptay\Dropbox\adiuvo\mission_critical_course\RTL\Synplify_Unsafe\synthesis\fsm.sap.  
  
Starting constraint checker (Real Time elapsed 0h:00m:00s; CPU Time elapsed 0h:00m:00s; Memory used current: 108MB peak: 110MB)  
  
Found Transition to recovery state 0 from every state  
Encoding state machine current_state[0:2] (in view: work.fsm(rtl))  
original code -> new code  
  00 -> 00  
  01 -> 01  
  10 -> 10  
@N:MO222 : | Using Safe Case FSMs technique,FSM error recovery to default state is enabled for current_state[0:2]  
None  
None
```

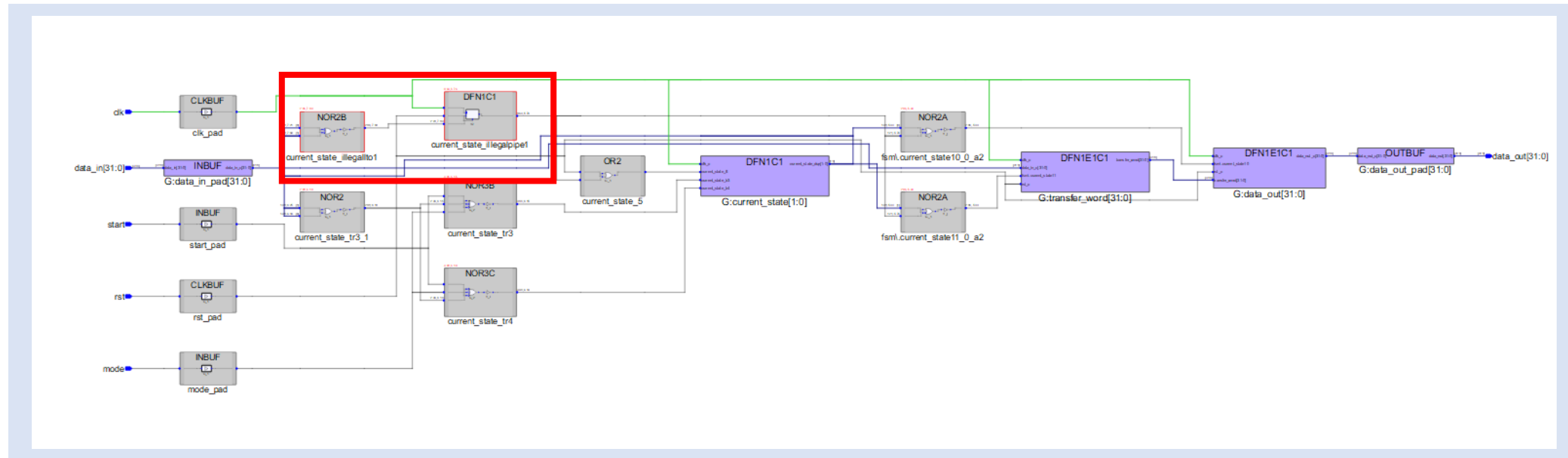
Libero FSM: Overview and Introduction to Libero

Step 9 – Open the RTL View notice the Error Detection Logic has been implemented



Libero FSM: Overview and Introduction to Libero

Step 10 – Open the Technology view and notice the insertion of illegal state detection logic.



Libero FSM: Overview and Introduction to Libero

Step 11 – In fsm_full_case.vhd comment out the state assignment in the when others clause – Re Run Synthesis

```
fsm : process (rst, clk)
begin
  if rst = '1' then
    current_state <= idle;
    transfer_word <= (others => '0');
    data_out <= (others => '0');
  elsif rising_edge(clk) then
    case current_state is
      when idle =>
        if start = '1' then
          if mode = '0' then
            current_state <= rd;
          else
            current_state <= wr;
          end if;
        end if;
      when rd =>
        data_out <= transfer_word;
        current_state <= idle;
      when wr =>
        transfer_word <= data_in;
        current_state <= idle;
      when others =>
        --current_state <= idle;
        --null;
    end case;
  end if;
end process;
```

Libero FSM: Overview and Introduction to Libero

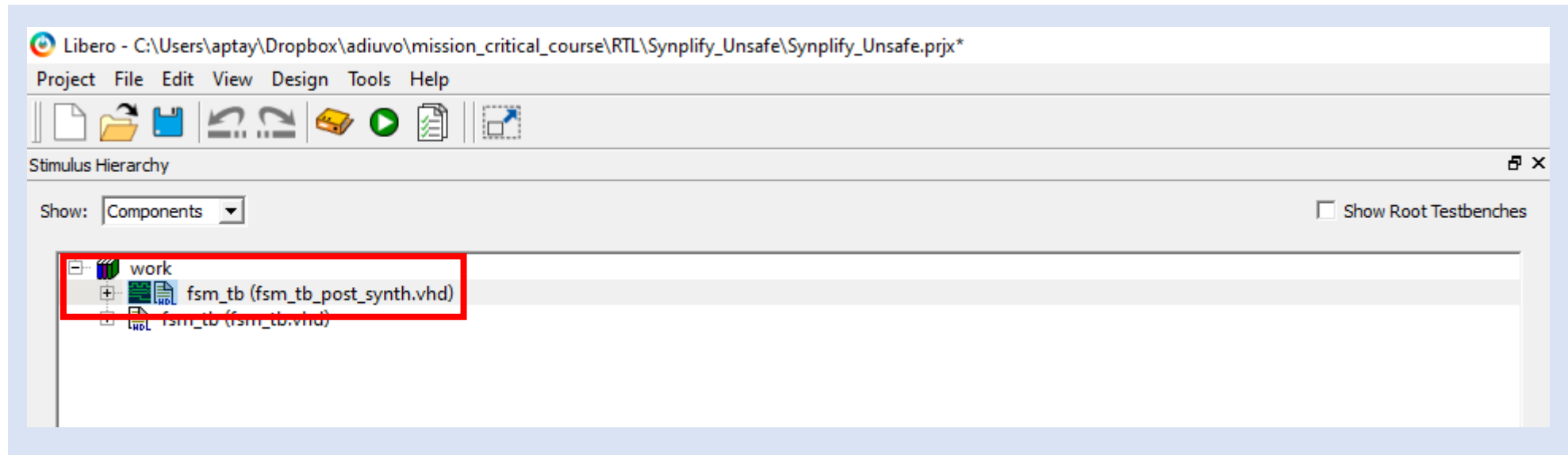
Step 12 – Examine the log and notice failure to implement safe state machine as the when other clause is do not care or null.

```
@N:CD720 : std.vhd(146) | Setting time resolution to ps
@N: : fsm_full_case.vhd(47) | Top entity is set to fsm.
File C:\Users\aptay\Dropbox\adiuvo\mission_critical_course\RTL\Synplify_Unsafe\hdl\fsm_full_case.vhd changed - recompiling
VHDL syntax check successful!
File C:\Users\aptay\Dropbox\adiuvo\mission_critical_course\RTL\Synplify_Unsafe\hdl\fsm_full_case.vhd changed - recompiling
@N:CD231 : stdl164.vhd(890) | Using onehot encoding for type mvl9plus. For example, enumeration 'U' is mapped to "1000000000".
@N:CD630 : fsm_full_case.vhd(47) | Synthesizing work.fsm.rtl.
@N:CD233 : fsm_full_case.vhd(65) | Using sequential encoding for type state.
Post processing for work.fsm.rtl
@W:CL269 : fsm_full_case.vhd(78) | State error detection not built
@N:CL275 : fsm_full_case.vhd(78) | Possible don't care specified as next state in others or default clause, or failure to extract state machine
At c_vhdl Exit (Real Time elapsed 0h:00m:00s; CPU Time elapsed 0h:00m:00s; Memory used current: 72MB peak: 73MB)
```

Lesson Objective – When creating High Reliable State Machines for Libero make sure the code implements the when others clause correctly.

Libero FSM: Overview and Introduction to Libero

Step 12 – To the stimulus hierarchy add in the file fsm_tb_post_synth.vhd set it to the active stimulus.



Libero FSM: Overview and Introduction to Libero

Step 13 – In your editor of choice open the file `fsm_tb_post_synth.vhd` and examine it. Note how the file uses VHDL-2008 features to be able to set the state of signals in the UUT to invoke an SEU type behavior.

```
stim : process
begin
  rst    <= '1';
  data_in <= (others => '0');
  start  <= '0';
  mode   <= '0';
  wait until rising_edge(clk);
  rst <= '0';
  wait for 1 us;
  wait until rising_edge(clk);
  start <= '1';
  mode  <= '1';
  data_in <= x"55aa55aa";
  wait until rising_edge(clk);
  start <= '0';
  wait for 1 us;
  wait until rising_edge(clk);
  start <= '1';
  mode  <= '0';
  wait until rising_edge(clk);
  start <= '0';
  mode  <= '0';
  wait for 1 us;
  wait until falling_edge(clk);
  << signal .fsm_tb.fsm_inst.\current_state[0]\.Q: std_logic >> <= force '1';
  << signal .fsm_tb.fsm_inst.\current_state[1]\.Q: std_logic >> <= force '1';
  wait until falling_edge(clk);
  << signal .fsm_tb.fsm_inst.\current_state[0]\.Q: std_logic >> <= release;
  << signal .fsm_tb.fsm_inst.\current_state[1]\.Q: std_logic >> <= release;
  wait for 1 us;
  report "simulation complete" severity failure;
end process;
```

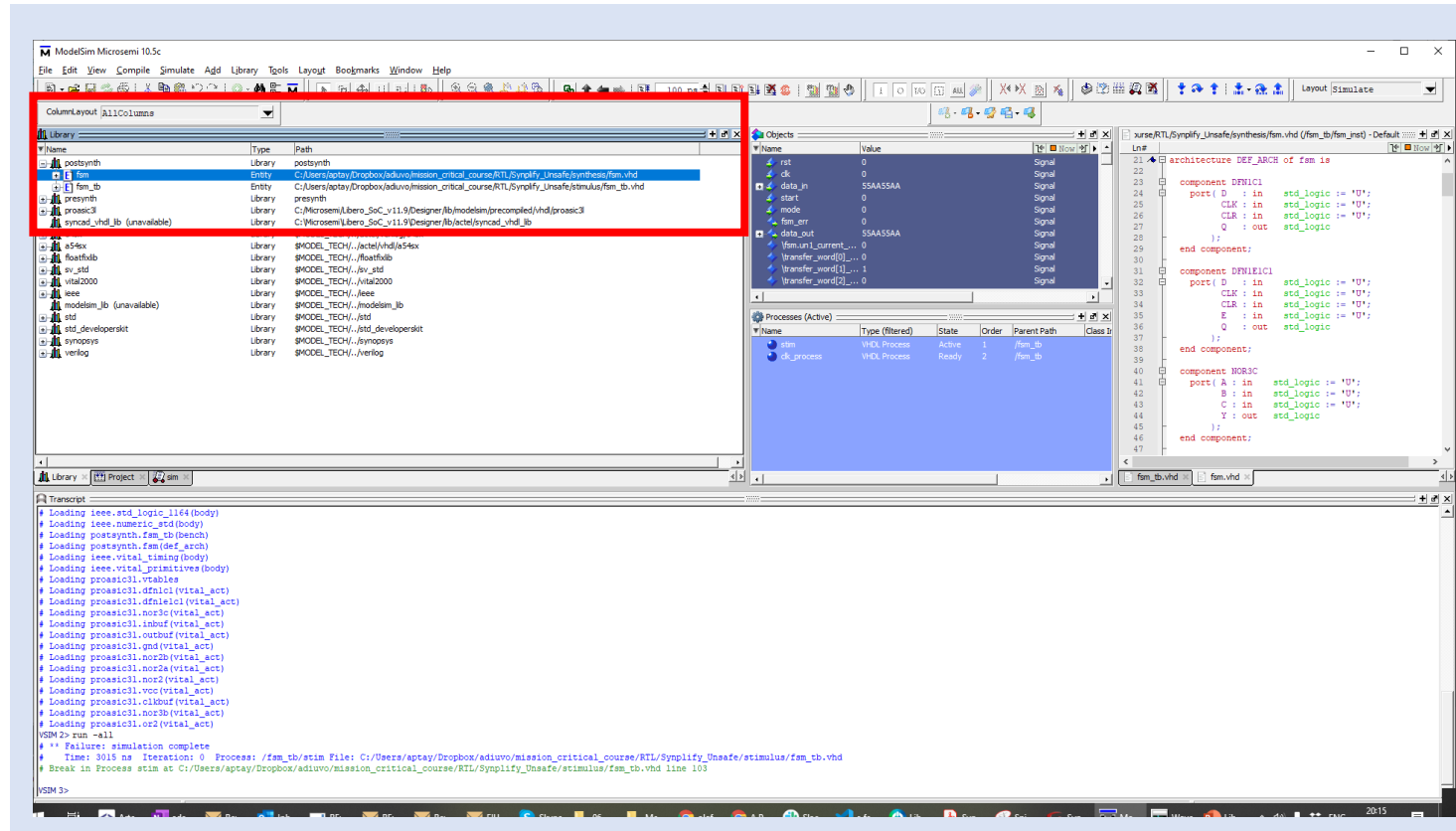
Hierarchical Access to the current_state signals

Force command enables signal to be driven and not have multiple driver issues.

Release command frees signal to enable design to resume driving the signal
Important to show the correction mechanism in place

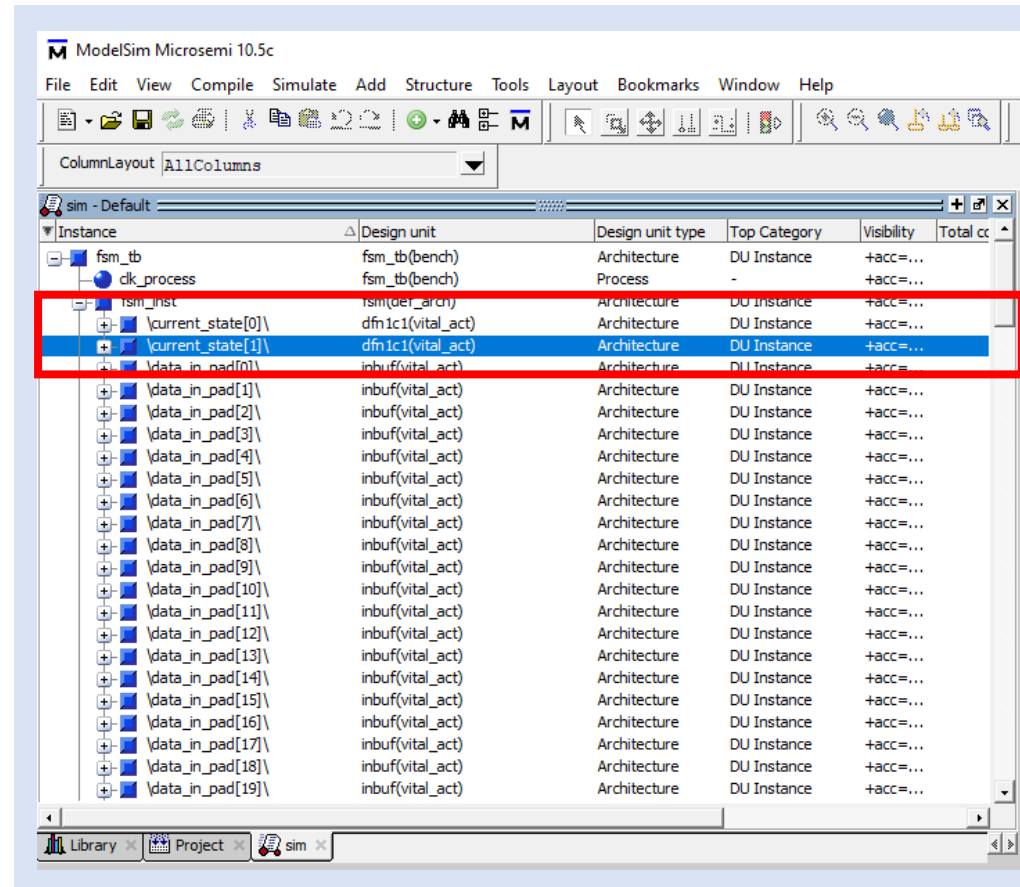
Libero FSM: Overview and Introduction to Libero

Step 14 – Back in Libero, run the Post Synthesis simulation Interactively. In ModelSim on the library tab look under post synth and examine the two files. Notice the FSM.vhd which is output from Synplify



Libero FSM: Overview and Introduction to Libero

Step 15 – From the SIM tab, add the `\current_state[0]` \ `\current_state[1]` \ `current_state_illegalto1` and `current_state_illegalpipe1` to the waveform view, run the simulation and observe the resultant waveform.



Libero FSM: Overview and Introduction to Libero

Step 16 – Looking at the end of the simulation you should see the corruption of the current_state Q outputs. Along with seeing the resultant detection of the illegal “11” state and the assertion of the CLR signals to reset the FSM back to the idle state.

